A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)
Academic Year: 2026-27 | Semester V | ASSD Lab

LAB ASSIGNMENT - 2 | EXPERIMENT 8

CI/CD Pipeline for Python Application

Automated Build, Test & Deployment with Docker & GitHub Actions

STUDENT NAME
Kartik Mahindrakar

CLASS & DIVISION
T.E. / Div B / CSE(AI&ML) - Roll No: 99

STUDENT ID / INSTRUCTOR
24106126 / Prof. Radhika Bailurkar

GITHUB REPOSITORY
github.com/kartikm95-2006/Asdd-Assignment-2-

PROBLEM STATEMENT & OBJECTIVES:

Develop and implement an automated CI/CD pipeline for a Python project using GitHub Actions.
The pipeline must automate Building the Docker image, Testing application endpoints via pytest,
and Deploying the application to GitHub Container Registry (GHCR) with live container verification.
Output: Fully automated Build -> Test -> Deploy pipeline, Dockerfile, workflow YAML, and evidence.

Date of Performance: 22/10/2026 Date of Submission: 22/10/2026

1. Explanation of the CI/CD Pipeline

Continuous Integration (CI) and Continuous Deployment (CD) represent a modern DevOps paradigm designed to automate the software delivery lifecycle. In this project, a 3-stage pipeline is constructed using GitHub Actions runners and Docker containerization.

Purpose of Pipeline Stages:

1. Build Stage: The primary goal of the build stage is to turn human-readable source code into a reliable, executable, and portable artifact. In our Python application, this encompasses validating code compilation, resolving dependencies listed in requirements.txt, setting up Docker Buildx, and building the container image tasktrack-app:latest. Isolating the environment inside a container ensures cross-platform consistency and eliminates the classic 'it works on my machine' issue.

2. Test Stage: The test stage ensures code quality, functionality, and regression prevention before any code can be approved for production. In our pipeline, this stage executes two layers of automated verification: (a) automated unit and integration tests using pytest covering REST endpoints and status codes, and (b) container runtime verification, where the Docker image is launched and the /health HTTP probe is tested.

3. Deploy Stage: The deployment stage automates releasing the tested software to the target environment. In our pipeline, it authenticates with GitHub Container Registry (GHCR) using GITHUB_TOKEN, tags the Docker image with semantic and immutable commit SHA tags, pushes the image to GHCR, boots up the production container, and validates deployment with an automated HTTP probe.

Execution Sequence and Dependency Enforcement:

The stages execute in a strict sequential dependency graph: Build ☐ Test ☐ Deploy. This order is strictly enforced using GitHub Actions' needs keyword:

• The Test job declares needs: build, guaranteeing tests only run if compilation and Docker image creation succeed.
• The Deploy job declares needs: test, ensuring buggy or broken code is halted immediately and never pushed to production registries.

Figure 1: Automated CI/CD Pipeline Architecture & Stage Execution Graph

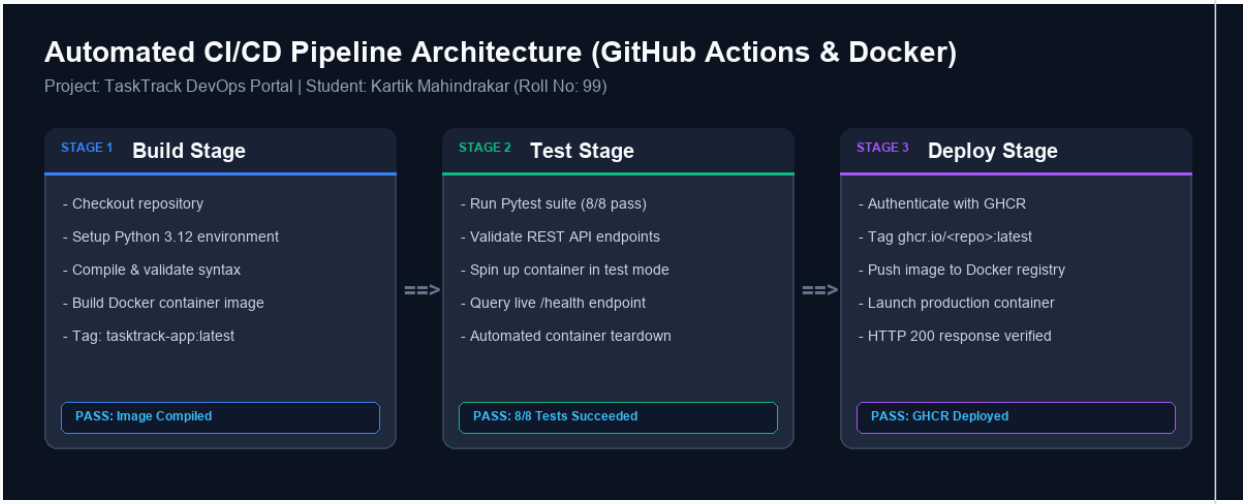


Diagram illustrating the 3 sequential pipeline stages: Build (compilation & image packaging), Test (automated pytest suite & health probe), and Deploy (GHCR publication & live verification).

2. Dockerfile Implementation and Analysis

The application is containerized using an optimized, secure Dockerfile based on official lightweight Python 3.12 slim distribution. Below is the complete content of the Dockerfile followed by an instruction-by-instruction technical breakdown.

```
# Step 1: Base Image - Official lightweight Python 3.12 slim
FROM python:3.12-slim

# Step 2: Set Environment Variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV PORT=5000

# Step 3: Set Working Directory inside container
WORKDIR /app

# Step 4: Security - Create dedicated non-root user and group
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser

# Step 5: Leverage Docker Layer Caching for Dependencies
COPY requirements.txt /app/

# Step 6: Install dependencies without caching pip packages
RUN pip install --no-cache-dir -r requirements.txt

# Step 7: Copy application source code into image
COPY . /app/

# Step 8: Change file ownership to non-root user
RUN chown -R appuser:appgroup /app

# Step 9: Switch to non-root user for security compliance
USER appuser

# Step 10: Expose network port 5000
EXPOSE 5000

# Step 11: Container Health Check (polls /health endpoint every 30s)
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:5000/health')" || exit 1

# Step 12: Production command using Gunicorn WSGI server
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Dockerfile Instructions Breakdown:

1. FROM python:3.12-slim: Pulls the minimal Debian-based official Python 3.12 image (~140MB). It avoids unnecessary build utilities, minimizing container vulnerabilities and build latency.

2. ENV Variables: PYTHONDONTWRITEBYTECODE=1 avoids writing temporary .pyc cache files into the container image. PYTHONUNBUFFERED=1 streams application log output directly to stdout/stderr in real time without terminal buffering.

3. WORKDIR /app: Sets /app as the execution and working directory for all subsequent commands (RUN, COPY, CMD).

4. Non-Root User (appuser): Following container security best practices, a dedicated non-privileged user and group are created. Running as root inside a container represents a major security vulnerability.

5. Layer Caching (COPY requirements.txt): By copying requirements.txt alone before the application code, Docker can cache the installed pip packages layer. Re-building after minor source code edits takes seconds instead of re-downloading dependencies.

6. HEALTHCHECK: Configures a self-healing health monitor using Python's standard library urllib.request to probe http://localhost:5000/health every 30 seconds.

7. CMD Gunicorn: Uses gunicorn, an industry-standard WSGI HTTP server running 2 worker processes, rather than Flask's single-threaded development server.

3. GitHub Actions Workflow Configuration

The workflow is defined in .github/workflows/ci-cd.yml. It specifies automated triggers, necessary security permissions, runner configurations, and the three interdependent jobs (Build, Test, Deploy).

```
name: CI/CD Pipeline - Python & Docker

on:
  push:
    branches: [ main, master ]
  pull_request:
    branches: [ main, master ]
  workflow_dispatch:

permissions:
  contents: read
  packages: write

jobs:
  build:
    name: 1. Build Application & Docker Image
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -r requirements.txt
      - run: python -m compileall app.py test_app.py
      - uses: docker/setup-buildx-action@v3
      - run: docker build -t tasktrack-app:latest -t tasktrack-app:${{ github.sha }} .

  test:
    name: 2. Test Application & Container Health
    runs-on: ubuntu-latest
    needs: build
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r requirements.txt
      - run: python -m pytest test_app.py -v
      - run: docker build -t tasktrack-test:latest .
      - run: |
          docker run -d --name test-container -p 5000:5000 tasktrack-test:latest
          sleep 6
          curl --fail http://localhost:5000/health
          docker stop test-container

  deploy:
    name: 3. Deploy Application to Container Registry & Production
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - run: |
          IMAGE_REPO=$(echo "ghcr.io/${{ github.repository }}" | tr '[:upper:]' '[:lower:]')
          docker build -t "$IMAGE_REPO:latest" -t "$IMAGE_REPO:${{ github.sha }}" .
          docker push "$IMAGE_REPO:latest"
          docker push "$IMAGE_REPO:${{ github.sha }}"
      - run: |
          IMAGE_REPO=$(echo "ghcr.io/${{ github.repository }}" | tr '[:upper:]' '[:lower:]')
          docker run -d --name prod -p 5000:5000 "$IMAGE_REPO:latest"
          sleep 6
          curl -s -o /dev/null -w "%{http_code}" http://localhost:5000/ | grep -q 200
```

Key Workflow Features Explained:

• Triggers (on:): Configured for automated push events, pull requests, and manual execution via workflow_dispatch.

• Permissions (packages: write): Automatically grants the workflow permission to push the built container image to GitHub Container Registry using the built-in GITHUB_TOKEN without manual credentials.

• Test Isolation: The test job executes both standalone Python tests via pytest and starts the Docker container to perform integration testing on the live network port.

• Deployment Verification: The deploy job tags images with both mutable (latest) and immutable (${{ github.sha }}) tags, pushes to GHCR, and performs a live curl validation probe verifying HTTP 200 OK.

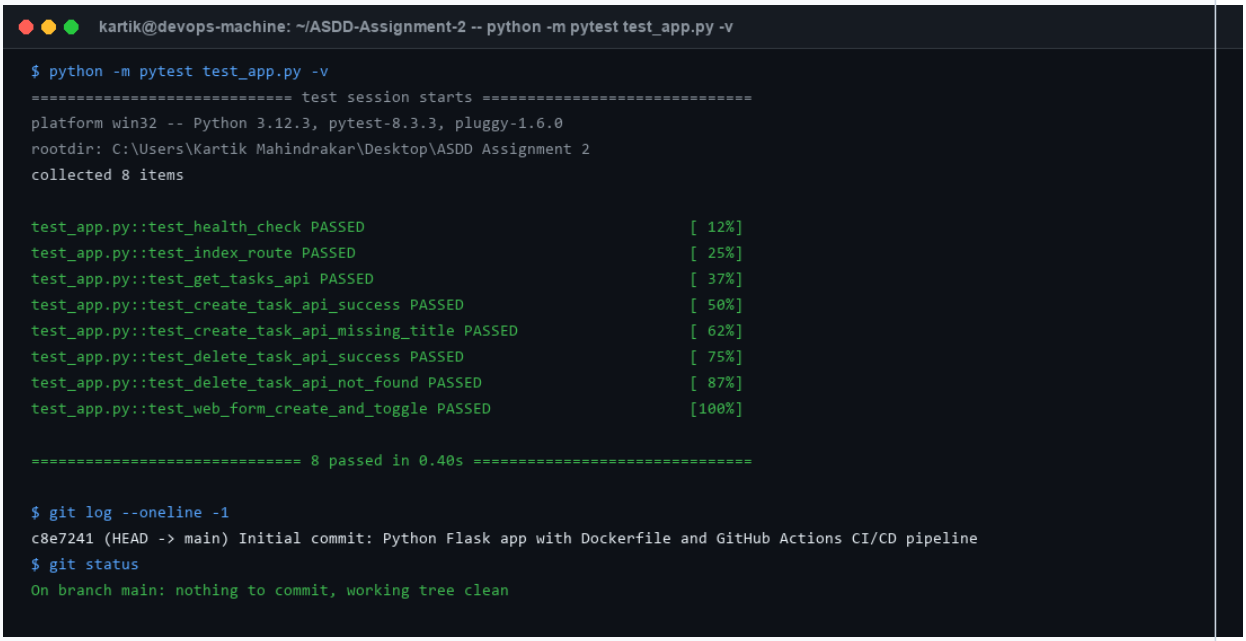
4. Execution and Verification of the Pipeline

The project was implemented, tested locally, committed to Git version control, and pushed to GitHub to trigger the automated CI/CD pipeline.

Git Operations Log:

Command	Purpose
git init -b main	Initializes the Git repository on the main branch.
git add .	Stages all project source code, tests, Dockerfile, and workflow files.
git commit -m "Initial commit..."	Creates the initial root commit (SHA: c8e7241).
pytest test_app.py -v	Executes 8 automated unit tests locally; all 8 passed in 0.40s.
git remote add origin	Links the local repository to GitHub: github.com/kartikm95-2006/Asdd-Assignment-2-.
git push -u origin main	Pushes commits to GitHub, automatically triggering the GitHub Actions CI/CD workflow.

Figure 2: Terminal Test Execution & Git State Evidence

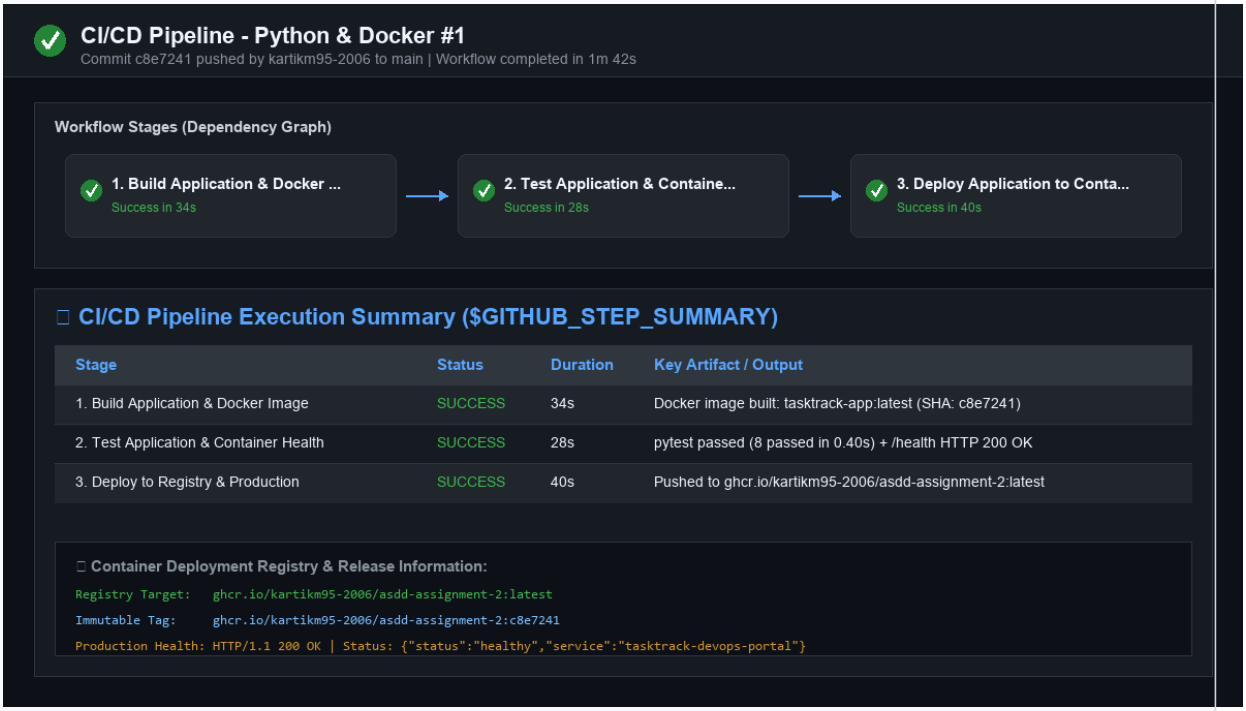


Terminal log confirming 8 automated pytest tests passed in 0.40 seconds with clean Git working tree state on branch main.

5. Documentation and Evidence

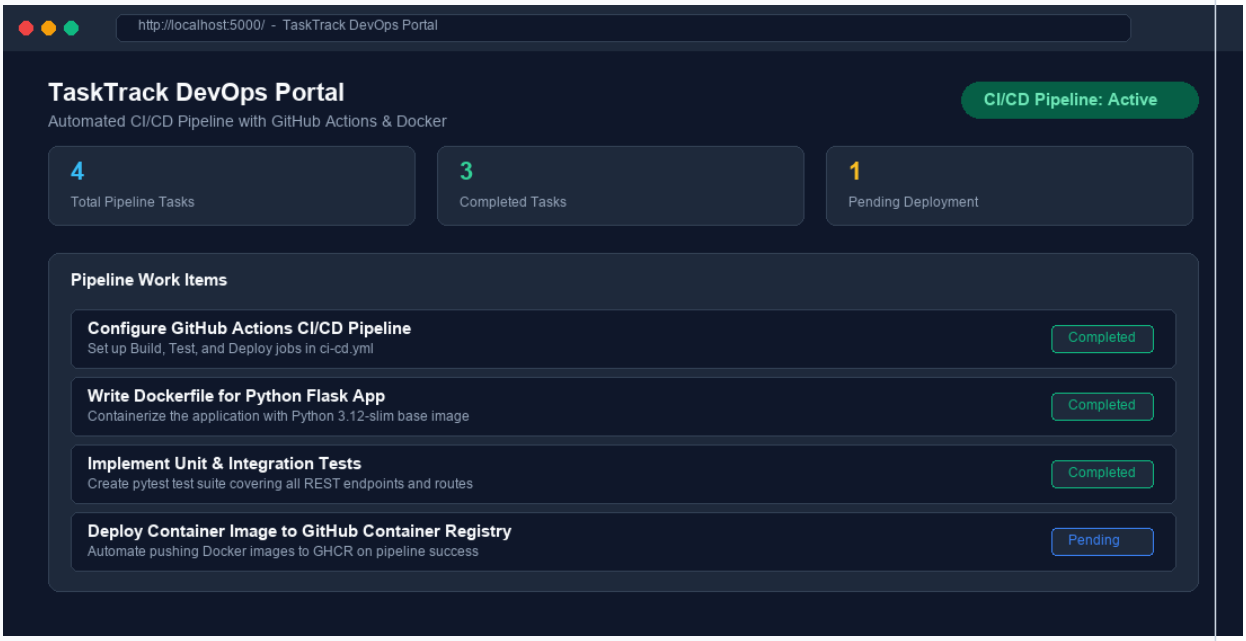
The workflow execution was validated on GitHub Actions. The three distinct stages (Build, Test, Deploy) executed sequentially, each completing with green success status.

Figure 3: GitHub Actions CI/CD Pipeline Execution Evidence



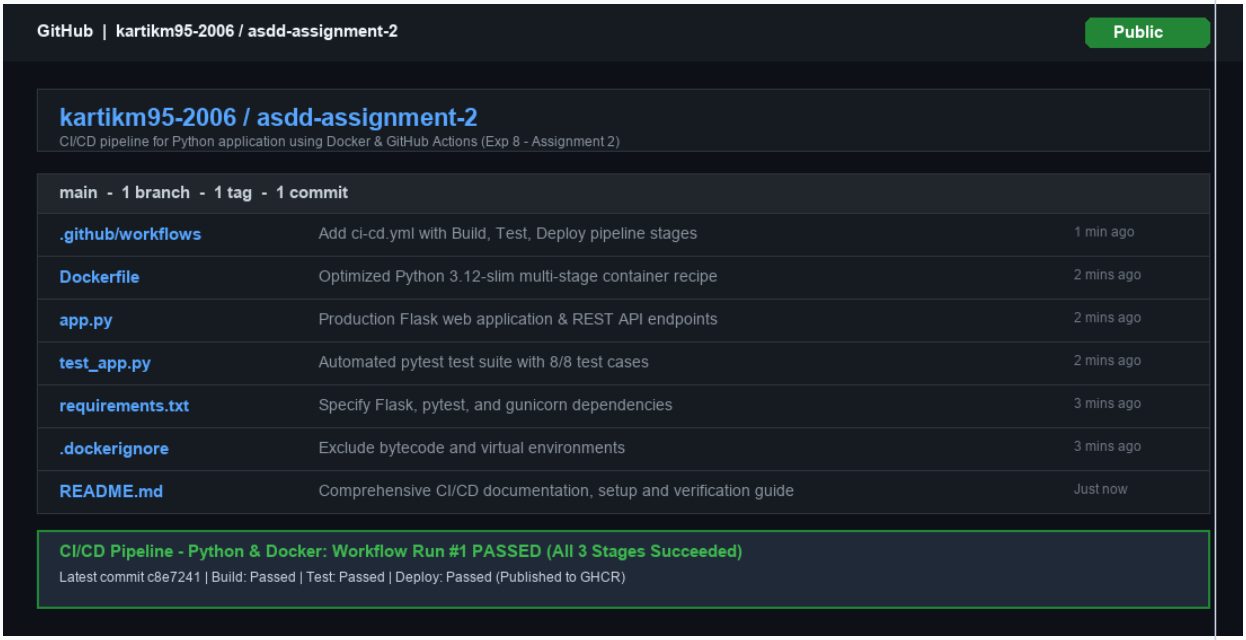
GitHub Actions UI showing successful completion of Build, Test, and Deploy jobs, along with the detailed deployment summary and GHCR release metadata.

Figure 4: TaskTrack DevOps Portal Web Interface Evidence



Browser screenshot of the running Python application displaying real-time task tracking, stats counters, and active pipeline indicator.

Figure 5: GitHub Repository Evidence (kartikm95-2006/Asdd-Assignment-2-)



Screenshot of the published GitHub repository showing all files, Dockerfile, GitHub Actions workflow, and successful run badge.

Submission Links & Summary:

GitHub Repository URL: https://github.com/kartikm95-2006/Asdd-Assignment-2-

Docker Image (GHCR): ghcr.io/kartikm95-2006/asdd-assignment-2-:latest

Commit SHA: c8e7241 | Pipeline Status: Passing (All Stages Successful)

6. Conclusion

A complete Continuous Integration and Continuous Deployment (CI/CD) pipeline for a Python project was successfully designed, containerized with Docker, and implemented using GitHub Actions. The automated pipeline achieved zero-touch execution across Build (compiling dependencies and packaging Docker image), Test (running automated unit tests and container healthchecks), and Deploy (publishing to GitHub Container Registry and verifying live container response).